



Interação entre Componentes do sistema

Você vai aprender a importância da identificação de requisitos na arquitetura de sistemas, como validar esses requisitos por meio da prototipação e definir interações entre componentes, avançar um desenvolvimento mais eficiente e alinhado aos resultados esperados.

Prof. Marcos Alexandre Fernandes Ferronato

Propósito

O domínio das metodologias de levantamento e validação de requisitos é fundamental para a formação do aluno, pois capacita profissionais da área de arquitetura de sistemas a desenvolver projetos mais eficazes e alinhados às necessidades dos usuários, garantindo que os resultados atendam às expectativas e aumentem a eficiência dos sistemas.

Objetivos

- Reconhecer a importância dos requisitos para melhorar o resultado na arquitetura de sistemas.
- Identificar a relação entre a identificação de componentes e o melhor resultado na arquitetura de sistemas.
- Analisar como são definidos e implementados os elementos de interação de componentes na arquitetura de sistemas.
- Verificar como os elementos de interação contribuem para o sucesso do projeto na arquitetura de sistemas.

Introdução

No desenvolvimento de sistemas, a forma como os requisitos são tratados pode impactar diretamente na qualidade e no sucesso do projeto. Existem abordagens que consideram os requisitos como elementos voláteis e, por isso, dedicam pouco tempo à sua definição inicial. Outras adotam o caminho oposto, investindo grande esforço para detalhá-los exaustivamente desde o início. Neste conteúdo, será apresentada uma alternativa intermediária, que equilibra a importância de se compreender bem os requisitos com a agilidade necessária ao projeto.

A proposta é entender que requisitos bem definidos são fundamentais, mas que esse processo deve ser realizado de maneira eficiente, ocupando no máximo 5% do tempo total do projeto. Para isso, será apresentada a técnica da prototipação como ferramenta para validação dos requisitos, otimizando tempo e aumentando a precisão na definição das necessidades do sistema.

A partir dessa base, o foco se volta para o processo de identificação de componentes, uma etapa essencial dentro das boas práticas de arquitetura de sistemas. Você será guiado por um conjunto de ações que permitem compreender como estruturar corretamente a interação entre os diversos módulos e componentes que compõem um sistema, garantindo coerência, escalabilidade e eficiência.

Além disso, será explorado como essa identificação influencia os resultados esperados e qual o papel do arquiteto de sistemas nesse processo. A correta definição das interfaces entre os componentes será um dos pontos centrais, demonstrando como esse cuidado contribui para a construção de modelos mais robustos e eficazes.

Com isso, você estará mais preparado para lidar com os desafios da modelagem por componentes, entendendo sua importância no contexto arquitetural e seu impacto direto nos resultados do projeto.

A importância dos requisitos

Podemos definir requisitos como condição ou exigência indispensável para um fim, uma ocupação e/ou uma realização.

Para começarmos é importante definirmos como as metodologias tratam requisitos e como essa disciplina fará a abordagem do tema, quais as diferenças encontradas no processo de levantamento de requisitos, no esforço gasto e em seus resultados, além de tratar da importância de requisitos para o resultado final do sistema.

Conheça a seguir as diferenças no levantamento dos requisitos!

Diagrama de funcionalidades de interface

Como dito anteriormente, algumas metodologias tratam requisitos de forma menor, pois entendem que, como mudarão até o fim do projeto, não vale a pena perder muito tempo com eles, nem dedicar muito esforço nesse trabalho.

Nesses casos, os usuários e a equipe de desenvolvimento são chamados para novas definições e validações em cada parte do sistema que fica pronta.

Lembrando que para essas metodologias, os modelos conceituais de negócios e de sistemas são suficientes para que todos os envolvidos possam, cada um dentro de sua área de conhecimento, validar o que foi feito até ali.

Isso não é uma verdade absoluta, pois dificilmente um usuário leigo vai conseguir validar requisitos frente a modelos conceituais (negócio e sistemas) feitos para os iniciados em arquitetura de sistemas.

Requisitos vistos com muita importância

Outras metodologias, ao contrário, entendem que os requisitos devem ser levantados completamente até a exaustão, independentemente do tempo necessário para isso.

Nesse caso, o processo se torna moroso, caro e muito frustrante para os usuários, já que eles têm de participar de intermináveis reuniões de definição, sem que os resultados sejam visíveis aos seus olhos.

O mesmo ocorre com a equipe de desenvolvimento, que acaba por gastar muita energia e foco em algo que demora a ficar consistente.

No caso dessas metodologias, a validação de requisitos é feita através de textos longos e cansativos que, dificilmente, levarão os usuários a um correto entendimento de seus sistemas e a uma correta e consistente validação dos requisitos.

Conceito intermediário

Como dissemos, nesta disciplina, usaremos um conceito intermediário, em que o esforço total no processo de requisitos não pode ultrapassar os 5% do esforço total do projeto.

Além disso, os requisitos devem ser validados com um mecanismo que seja eficiente e eficaz para esse fim.

Essas definições de esforço e dos mecanismos veremos mais detalhadamente nos itens a seguir.

Requisitos de sistemas

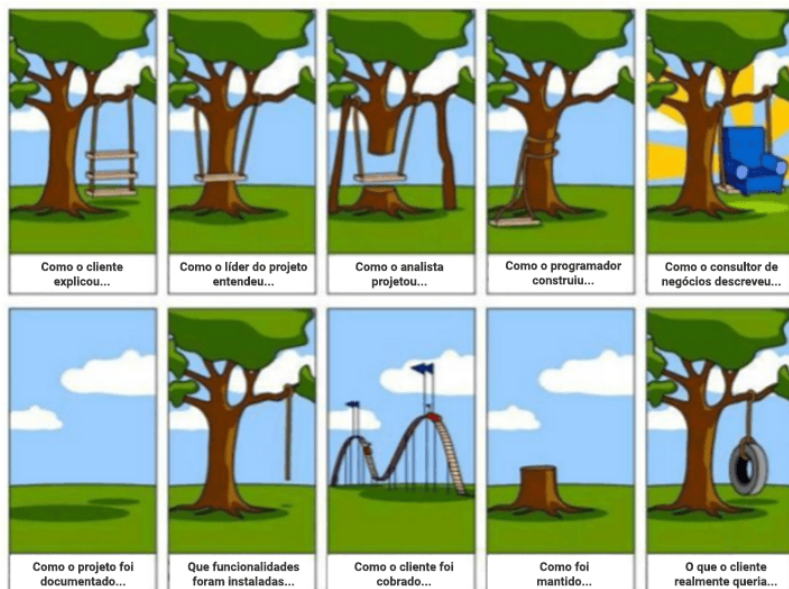
Os requisitos de sistemas fazem parte da família de requisitos e compartilham sua definição básica. Porém, estão associados a funcionalidades e tipos que veremos mais detalhadamente a seguir.

O conhecimento adquirido aqui servirá para que nossas partes interessadas e nossas equipes de desenvolvimento não enfrentem o problema apresentado no quadrinho.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Esforço máximo e resultados esperados na etapa de requisitos

É de se supor que gastar somente 5% do esforço do projeto em levantamento e validação de requisitos, como estamos propondo aqui nesta disciplina, nos leve a estar mais próximos das metodologias que dão pouca importância aos requisitos. Mas podemos garantir que isso está longe de ser verdade.

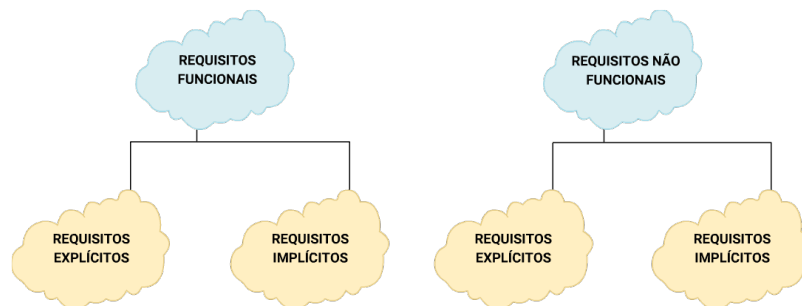
A delimitação de esforço máximo está associada a direcionar os trabalhos ao que realmente é importante para o levantamento, a validação de requisitos e também para o uso de ferramentas e técnicas baseadas em boas práticas de gestão e de desenvolvimento.

Tipos de requisitos



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Funcionais e não funcionais

Podemos separar os requisitos em duas categorias:

- Requisitos funcionais, que são aqueles que tratam do que o sistema deve fazer, de sua funcionalidade;
- Requisitos não funcionais, que tratam dos critérios que suportam os requisitos funcionais.

Como, por exemplo, qualidade para o software, os requisitos de performance, usabilidade, confiabilidade, robustez, etc. Ou, então, os critérios relativos à qualidade para o processo de software ou uso de metodologias.

Implícitos e explícitos

Para os dois tipos de requisitos devemos, ainda, separá-los em:

- Requisitos implícitos, que são aqueles que o usuário vai conseguir externar e repassar à equipe de desenvolvimento;
- Requisitos implícitos, que são aqueles que o usuário não vai conseguir repassar, ou por falta de conhecimento ou por entender ser muito óbvio e julgar que não vale a pena falar.



Saiba mais

Para complementar as informações faltantes (requisitos implícitos), é necessário que a equipe de desenvolvimento tenha um framework de conhecimentos bem vasto. Dessa maneira, é capaz de preencher as lacunas, tanto para requisitos funcionais como para requisitos não funcionais.

Framework de conhecimentos em TI como base para requisitos

O framework de conhecimentos em TI tratado aqui refere-se aos conhecimentos que vão além dos requisitos, como, por exemplo:

- Infraestrutura de hardware.
- Banco de dados.
- Comunicação.
- Confiabilidade do sistema.
- Nível de criticidade das operações.
- Desempenho do sistema.
- Adequações legais e éticas.
Segurança.

Esses conjuntos de conhecimentos servirão para preencher as lacunas que ficarem após as reuniões de levantamento de requisitos.



Atenção

Note que é normal que as lacunas existam, pois os usuários não conhecem e nem precisam conhecer elementos acima. Esse é um papel que obrigatoriamente deve ser desempenhado pela equipe de desenvolvimento, dentro do contexto previsto na arquitetura de sistemas.

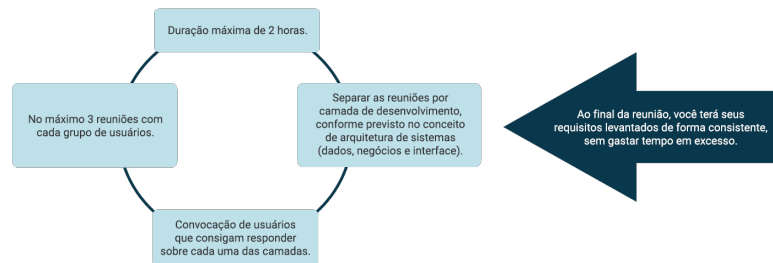
Reunião de levantamento de requisitos

Os trabalhos de levantamento e validação de requisitos deve conter uma reunião de levantamento de requisitos, com as seguintes características:



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Validação de requisitos

A validação de requisitos deve ser feita em duas etapas, chamadas de pré-validação e validação de requisitos.

Pré-validação de requisitos

Confira o passo a passo a seguir!

- Convoque uma reunião de 2 h com todas as partes interessadas do seu projeto;
- Separe as partes interessadas do projeto em grupos de 4 integrantes;
- Gere uma cópia do documento de requisitos para cada um dos grupos;
- Cada grupo deve revisar o documento por no máximo 15 minutos, com as observações que acharem pertinentes;
- Fazer com que todos os grupos revisem todas as cópias dos documentos.

Ao final desses passos, você terá seus requisitos validados por todas as partes interessadas no projeto. Além disso, conseguirá o apoio e o patrocínio necessário, pois todos são responsáveis pelo documento gerado e não somente a equipe de desenvolvimento.



Saiba mais

A nomenclatura técnica desta técnica é “Técnica de Delphi”.

Prototipação

A segunda etapa é desenvolvida dentro da prototipação que veremos detalhadamente agora!

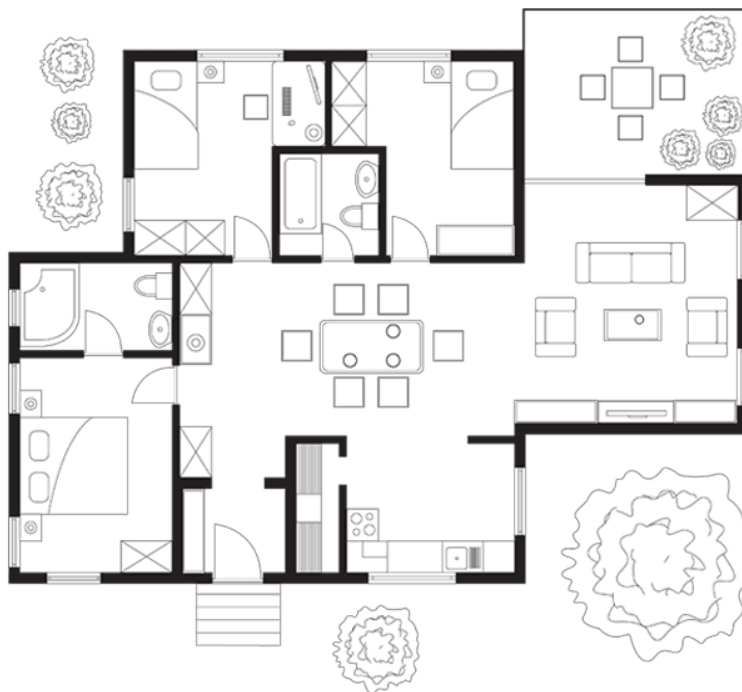
A prototipação é a melhor maneira de se validar requisitos.

Com essa técnica, conseguimos criar um modelo conceitual forte para ser validado pelas partes interessadas.

Mas o que é um conceito de modelo conceitual forte?

A melhor maneira de responder isso é fazer um comparativo entre modelos conceituais de outras áreas de conhecimento que tenha modelos conceituais fortes, como os modelos conceituais de negócio e de sistemas. A partir dessa comparação, fica mais fácil entender o conceito proposto.

Por exemplo, se usarmos como base para exemplificar um modelo conceitual forte, podemos partir da arquitetura e de uma planta de casa:



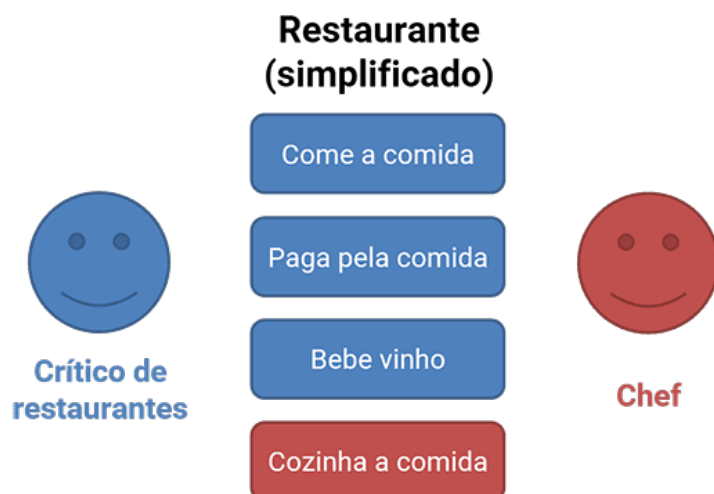
Não precisamos ser arquitetos para conseguir entender o conceito proposto a partir da planta (modelo). Isso representa a definição de um modelo conceitual forte.

E o modelo conceitual fraco?



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Quando olhamos para os diagramas de casos de uso, ou de classes, de objetos ou de componentes, verificamos que é necessário ser iniciado na arquitetura de sistemas para entender e validar se estão em conformidade com as necessidades do sistema a ser desenvolvidos (requisitos).

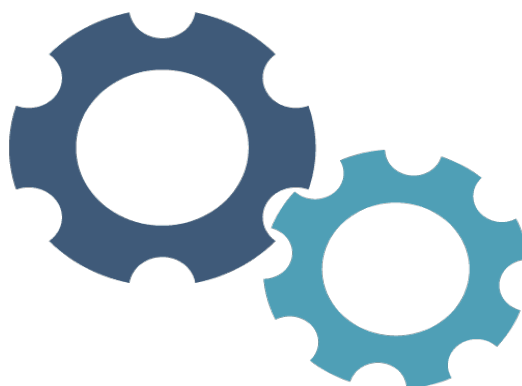
Isso representa um modelo conceitual fraco, ou seja, precisa de explicação externa para ser entendido.

Como criar para a TI um modelo conceitual forte?

Como, então, criar para área de TI um modelo conceitual forte, que seja facilmente entendido pelas partes interessadas e, ao mesmo tempo, esteja em adequação às boas práticas da arquitetura de sistemas?

A resposta está na prototipação de sistemas!

É com o protótipo que as partes interessadas vão conseguir facilmente verificar se seus requisitos estão presentes na definição do que será feito.



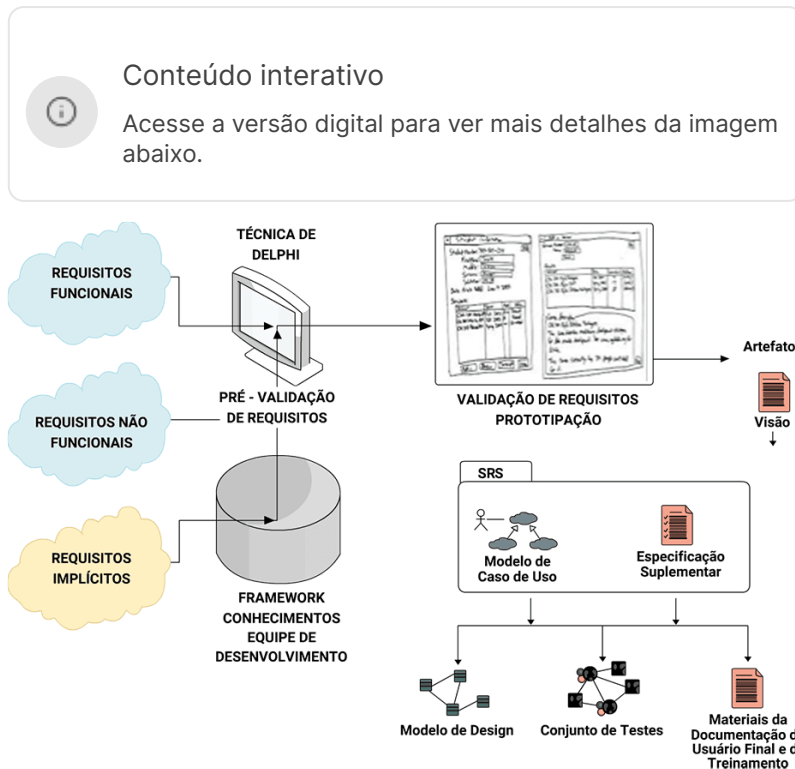
Atenção

É importante deixar claro às partes interessadas leigas que o protótipo é somente a casca e serve para validar funcionalidade e que ainda existe muito trabalho a ser feito para que o sistema fique pronto.

Modelagem conceitual de negócios e de sistemas a partir dos requisitos validados

Agora que temos todos os requisitos levantados e validados de forma consistente e confiável pelas partes interessadas, podemos iniciar os trabalhos de modelagem conceitual de negócios e de sistemas.

Essa modelagem será feita de acordo com os elementos estudados anteriormente.



Resultados esperados

Como resultado final desse conjunto de processos, espera-se que todas as necessidades das partes interessadas estejam presentes no sistemas desenvolvidos, na forma de funcionalidades, restrições e uso de tecnologias, atendendo plenamente a tudo o que se espera do novo sistema.

É de responsabilidade da equipe de desenvolvimento o atingimento dos resultados esperados.



Verificando o aprendizado

Questão 1

A gestão de requisitos é uma etapa importante do projeto por ser um conjunto de atividades que tem como principal objetivo ajudar a equipe de projeto a:

- A Utilizar ferramentas de engenharia de software para modelar os requisitos do sistema, através da UML.
- B Identificar, controlar e rastrear requisitos e modificações de requisitos em qualquer época, à medida que o projeto prossegue.

C Construir um modelo técnico refinado de funções, características e restrições do software.

D Negociar com os clientes os conflitos de prioridade de requisitos e identificar e analisar os riscos associados a cada requisito.

E Avaliar os requisitos quanto à qualidade, garantindo que ambiguidades, inconsistências, omissões e erros tenham sido detectados e corrigidos.



A alternativa B está correta.

É importante a gestão de requisitos em identificar, controlar e rastrear requisitos e mudanças ao longo do projeto.

Questão 2

Os requisitos não funcionais normalmente surgem por meio das necessidades dos usuários, podem ser restrições de orçamento, políticas organizacionais ou mesmo por fatores externos, como regulamentos de segurança e legislações de privacidade. Juntamente com a classificação dos requisitos não funcionais estão os requisitos de produto, os quais:

A Especificam ou restringem o comportamento do software, incluindo requisitos de desempenho, especificações de rapidez de execução e requisitos de confiabilidade que estabelecem, por exemplo, a taxa aceitável de falhas.

B São os requisitos gerais de sistemas derivados das políticas e procedimentos da organização do cliente e do desenvolvedor, como, por exemplo, os requisitos de processo operacional.

C Definem os requisitos do processo de desenvolvimento, como, por exemplo, a linguagem de programação, o ambiente de desenvolvimento ou normas do processo a serem usadas.

D Abrangem todos os requisitos que derivam de fatores externos ao sistema e seu processo de desenvolvimento. Podem incluir requisitos reguladores, que definem o que deve ser feito para que o sistema seja aprovado para uso.

E Incluem os requisitos legais, que devem ser seguidos para garantir que o sistema opere dentro da lei, e os requisitos éticos, que asseguram que o sistema será aceitável para seus usuários e o público geral.



A alternativa A está correta.

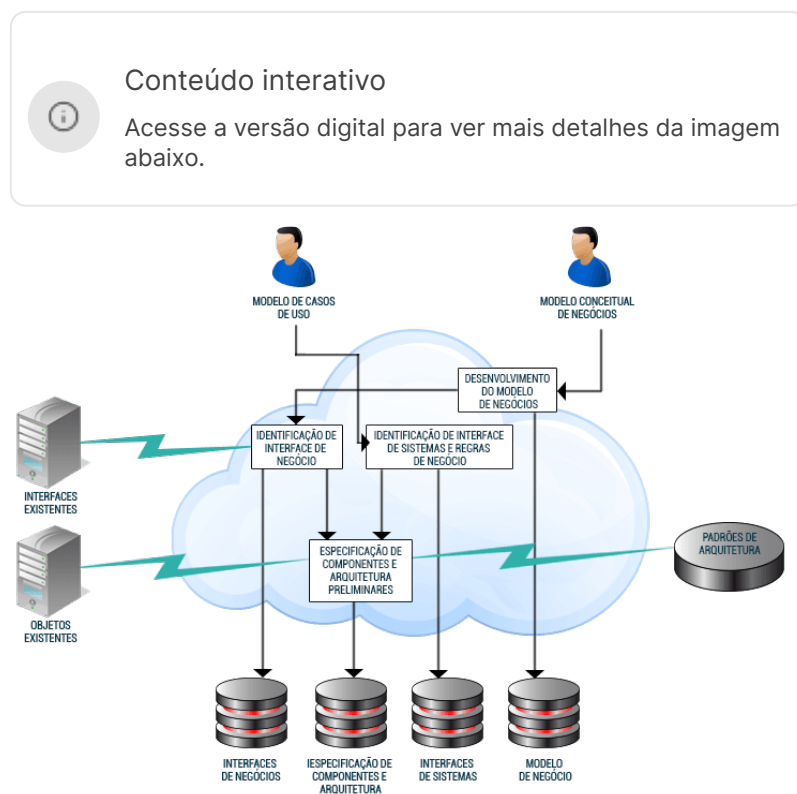
Os requisitos de produto descrevem e restringem o comportamento do software, incluindo aspectos como desempenho, rapidez, confiabilidade e taxas aceitáveis de falhas.

A importância da identificação de componentes

A identificação de componentes é o primeiro processo da metodologia apresentada por esta disciplina, que está baseada nas boas práticas da arquitetura de sistemas.

Seu objetivo é criar uma visualização inicial de todos os elementos envolvidos (modelos, métodos, objetos, componentes, etc.) e como eles são integrados.

Na imagem, você vê graficamente a origem dessas informações e suas ligações:



Os **artefatos** gerados a partir desse primeiro processo são:

- Interface de negócios.
- Interface de sistemas.
- Especificação de componentes e arquitetura do sistema.
- Modelo de negócios.

Perguntas associadas aos modelos de negócios

Precisamos, primeiramente, separar o modelo de negócios em grandes grupos que estão associados às seguintes perguntas:

Como?

Definem os recursos-chaves, as atividades-chaves e os componentes prontos que podem ser utilizados no novo sistema.

Para quem?

Definem os principais usuários do sistema, quais os seguimentos de atuação do cliente que ele rodará e quais elementos de infraestrutura devem ser utilizados pelo sistema a ser desenvolvido.

O quê?

Define qual ou quais problemas o sistema deve resolver depois que ficar pronto.

Quanto?

Informações sobre custo computacional e criticidade do sistema e quais benefícios se espera alcançar com o sistema a ser desenvolvido.

Artefatos e subprocessos de desenvolvimento do modelo de negócios

Veja a seguir a definição dos artefatos, como cada subprocesso é desenvolvido e o que devem conter cada um dos artefatos gerados.

Artefato modelo de negócios

Baseado no modelo CANVAS de modelagem de negócios, é gerado a partir do molde proposto pela metodologia que estamos aplicando nesse conteúdo.

Como		O que	Para quem	
Principais sistemas e elementos prontos que podem fornecer componentes	Atividades chaves do sistema a ser desenvolvido	Que problema o sistema proposto vai resolver	Principais usuários do sistema a ser desenvolvido	Qual(s) o(s) segmento(s) de atuação do cliente e como são atendidos no contexto atual
	Recursos chaves a serem utilizados pelo sistema a ser desenvolvido		Elementos de infraestrutura de hardware, BD, comunicação e software a ser utilizado pelo sistema a ser desenvolvido	
Qual o custo computacional previsto e a criticidade do novo sistema		Quais os benefícios se espera alcançar como o novo sistema		

Como	O que	Para quem
Quanto		

Tabela: Modelo para desenvolvimento do modelo de negócios - arquitetura de sistemas (baseado em canvas).
Marcos Alexandre Fernandes Ferronato.

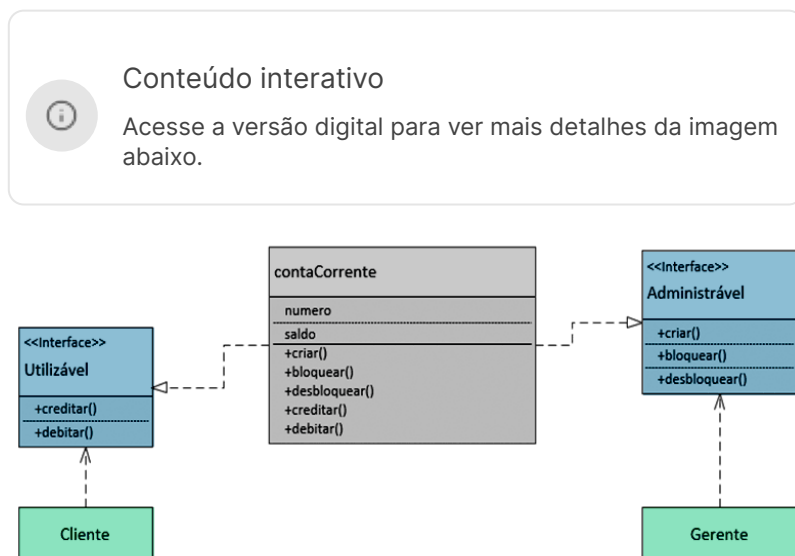
Subprocesso identificação de interfaces de negócio

Este subprocesso tem como objetivo a identificação, via modelo, das interfaces que mapearão as funcionalidades do sistema em relação à implementação do negócio a ser resolvido com o sistema a ser desenvolvido.

Artefato interface de negócios

O artefato gerado será um modelo que apresente todas as classes do sistema a ser desenvolvido e suas interfaces de utilização, mapeando como os atores farão os acessos ao sistema e que tipo de funcionalidade estará disponível para cada um deles.

Essas informações estão mapeadas no exemplo abaixo:



Subprocesso identificação de interfaces de sistemas e regras de negócio

Neste subprocesso, é necessário identificar como os componentes de sistema serão interligados de maneira a resolver o proposto no modelo de negócio, para o novo sistema a ser desenvolvido.

É necessário que todos os elementos estejam mapeados e suas ligações definidas, de maneira a se conseguir uma visão sistêmica entre os elementos e como eles resolvem as regras de negócio.

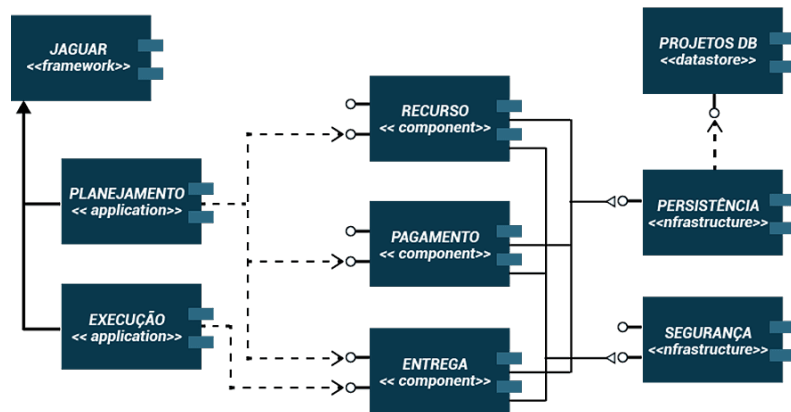
Artefato interface de sistemas

O artefato gerado aqui, como mostra a imagem, traduz a necessidade de identificação dos componentes disponíveis e como eles serão interligados e resolverão as regras de negócio, para o novo sistema a ser desenvolvido.

Conteúdo interativo



Acesse a versão digital para ver mais detalhes da imagem abaixo.



Subprocesso especificação de componentes e arquitetura do sistema

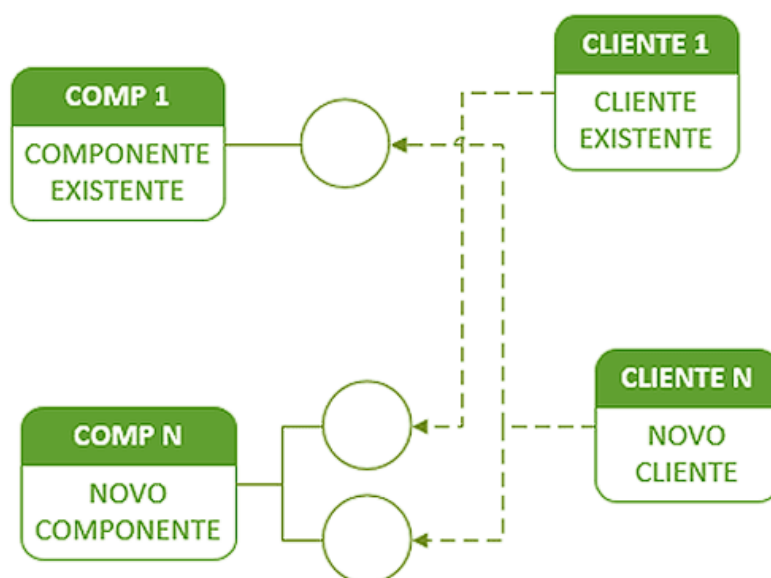
A partir da identificação das interfaces e da solução das regras de negócios, feitas no subprocesso anterior, é possível definir quais componentes já estão prontos e quais deverão ser desenvolvidos.

Todas essas informações devem estar representadas no artefato gerado a seguir.

Conteúdo interativo



Acesse a versão digital para ver mais detalhes da imagem abaixo.



Verificando o aprendizado

Sobre as características da engenharia de componentes, a alternativa INCORRETA é:

- A Os riscos são menores ao usar um componente já existente em relação ao desenvolvimento de algo a partir do zero.
- B Há aumento da produtividade, tendo em vista a redução de esforços pela equipe de desenvolvimento, seguindo a ideia “Crie uma vez, use onde quiser”.
- C A qualidade e confiabilidade do produto são maiores, pois o componente reutilizado já foi testado e aprovado.
- D Mesmo no caso de componentes comerciais, há sempre o acesso ao código-fonte e não há a preocupação com direitos autorais e licenças.

Resistência da parte das equipes de desenvolvimento, pois exige forte padronização (investimento de tempo e controle de qualidade) e documentação (investir mais tempo nos artefatos). A adoção de novas práticas de desenvolvimento geralmente encontra forte resistência a mudanças por parte da equipe.



A alternativa D está correta.

Componentes comerciais geralmente possuem restrições de acesso ao código-fonte, além de questões de direitos autorais e licenças.

Interface e interação de componentes

A modelagem de interação de componentes é uma técnica de modelagem do comportamento dos componentes em relação ao problema a ser resolvido.

Aqui, ela será usada para:

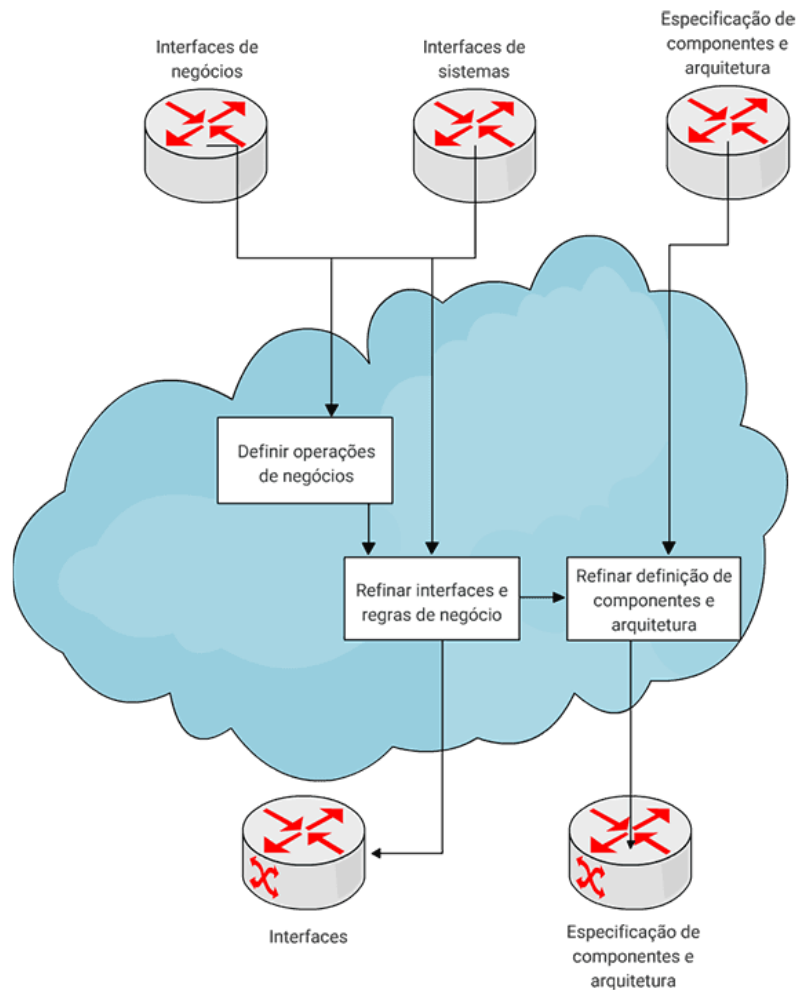
- Definir as várias interações que precisam ocorrer dentro do sistema;
- Refinar as definições de interface existente;
- Identificar como as interfaces serão utilizadas;
- Descobrir novas interfaces e operações.

Nesse contexto, você analisará trabalhos que abrangem a avaliação de várias interações e a definição de padrões comuns de utilização, identificando operações gerais que poderão substituir as dependências de interface específicas.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Ao final, você deverá reexaminar e refinar a arquitetura de componentes.

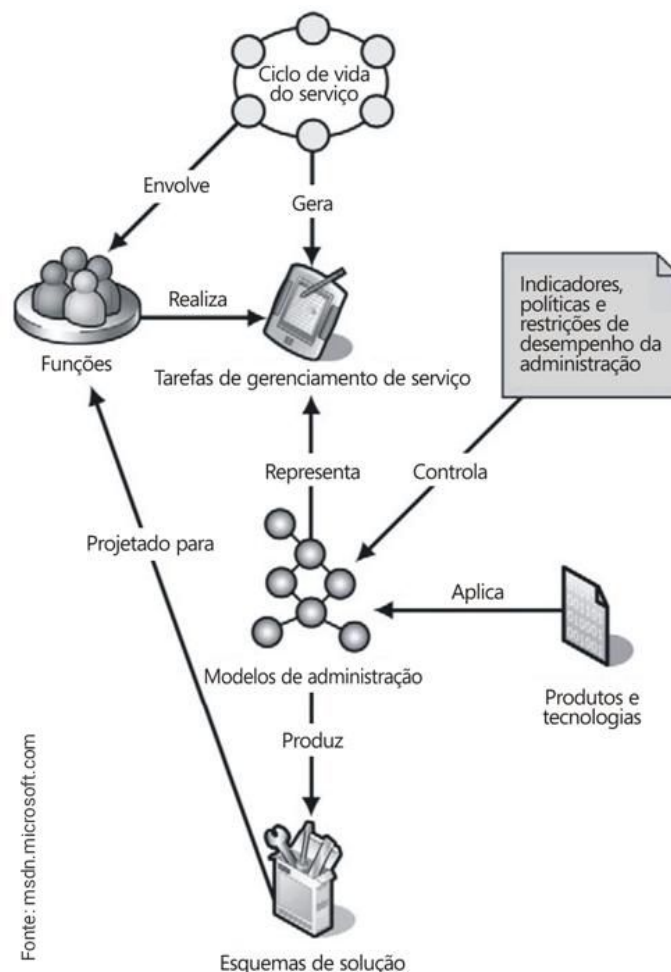
Definição das operações de negócios

O artefato de operações de negócio é a parte da modelagem de processos de negócios focada nas operações resultantes deste negócio, pois fornece uma solução clara e adaptável para capturar as especificações operacionais dos processos de negócio.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Neste momento, o modelo de negócios é validado e as operações são representadas, de maneira que sejam vistas todas as relações de funcionamento do negócio que o sistema proposto está disposto a resolver.

Refinar interfaces e regras de negócio

Refinar as regras de negócio é o subprocesso responsável por revistar o modelo de negócios, alterando e adaptando os elementos na medida das necessidades, já que agora temos as informações das operações de negócio mapeadas e definidas.

Como		O que	Para quem	
Principais sistemas e elementos prontos que podem fornecer componentes	Atividades chaves do sistema a ser desenvolvido	Que problema o sistema proposto vai resolver	Principais usuários do sistema a ser desenvolvido	Qual(s) o(s) segmento(s) de atuação do cliente e como são atendidos no contexto atual
	Recursos chaves a serem utilizados pelo sistema a ser desenvolvido		Elementos de infraestrutura de hardware, BD, comunicação e software a ser utilizado pelo sistema a ser desenvolvido	

Como	O que	Para quem
Qual o custo computacional previsto e a criticidade do novo sistema	Quais os benefícios se espera alcançar como o novo sistema	
Quanto		

Tabela: Modelo para desenvolvimento do modelo de negócios - arquitetura de sistemas (baseado em canvas).
Marcos Alexandre Fernandes Ferronato.

Refinar definição de componentes e arquitetura

Neste subprocesso também se espera que o arquiteto de sistemas faça um refinamento do modelo feito previamente, com as informações de operações e do modelo de negócios mapeadas e definidas de forma definitiva.



Neste contexto, vimos aqui os três subprocessos:

- Definir operações de negócios;
- Refinar interfaces e regras de negócio;
- Refinar definição de componentes e arquitetura.



Atenção

Isso serve como mecanismo de garantia de qualidade e de assertividade no resultado do novo sistema a ser desenvolvido, dentro do contexto da arquitetura de sistemas.

Verificando o aprendizado

Questão 1

Sobre interação de componentes, analise as sentenças a seguir, verifique quais são verdadeiras e depois marque a opção correspondente:

(I) Sempre que possível, a solução deve ser construída por meio de componentes já existentes, sendo eles componentes “comerciais de prateleira” (comercial off-the-shelf - COTS) ou componentes implementados dentro da empresa (in-house).

(II) A equipe de desenvolvimento deve tentar modificar ou remover os requisitos do sistema que não puderem ser implementados como componentes COTS ou desenvolvidos internamente.

(III) O desenvolvimento baseado em componentes é um tipo de desenvolvimento de software fortemente baseado no reuso.

A I e II estão corretas.

B I e III estão corretas.

C II e III estão corretas.

D I, II e III estão corretas.

E II está correta.



A alternativa D está correta.

A utilização de componentes existentes (COTS ou internos) é uma prática recomendada, a adaptação de requisitos pode ser necessária diante de limitações tecnológicas, e o desenvolvimento baseado em componentes visa maximizar o reuso de software.

Questão 2

Qual das seguintes afirmações esta correta:

A Na interação entre os componentes, é necessário descobrir as operações de negócio que estão ligadas ao relacionamento entre os componentes, através de diagramas dinâmicos, como de colaboração, de sequência ou de atividades.

B As operações de negócio ligadas ao relacionamento entre componentes são descobertas principalmente através da análise de documentos textuais e especificações estáticas.

C A identificação das operações de negócio entre componentes é um processo intuitivo que não requer o uso de modelos ou diagramas específicos.

D O foco principal na descoberta de operações de negócio entre componentes reside na análise individual de cada componente, sem considerar suas interações.

E As operações de negócio entre componentes só se tornam relevantes após a implementação completa do sistema.



A alternativa A está correta.

A identificação das operações de negócio entre componentes é feita por meio de diagramas dinâmicos, como os de colaboração, sequência ou atividades, que mostram a interação e o fluxo entre eles.

Questão 3

Na especificação final dos componentes, o Modelo de Informação das Interfaces (Interface Information Model) é utilizado porque:

(I) Provê a relação entre cada interface e as entidades do modelo de negócio.

(II) Ajuda o entendimento do contexto de cada interface.

(III) Mantém o conhecimento do domínio somente com o desenvolvedor responsável pela informação.

Das sentenças acima, qual está INCORRETA?

A I

B II

C III

D I e III

E II e III



A alternativa C está correta.

A sentença III está incorreta porque o Modelo de Informação das Interfaces visa compartilhar o conhecimento do domínio entre todos os envolvidos, e não restringi-lo a um único desenvolvedor.

Questão 4

Leia as informações a seguir:

Fases:

1 - Aquisição dos componentes;

2 - Localização de componentes prontos;

3 - Reutilização de componentes;

4 - Implementação dos Componentes;

Processos:

I - Pode ser necessário adaptar os componentes reutilizados ou até mesmo as funcionalidades do sistema (renegociação dos requisitos);

II - Reutilização de componentes prontos ou a utilização de novos componentes;

III - Busca por serviço fornecido pelo componente, considerando a semelhança de seus conteúdos;

IV- Deve-se utilizar um modelo de componente já existente, tais como EJB, COM+ etc.

Agora, veja qual a alternativa relaciona corretamente cada fase com seu processo é:

A 1-IV, 2-II, 3-III, 4-I

B 1-II, 2-III, 3-I, 4-IV

C 1-I, 2-II, 3-III, 4-IV

D 1-III, 2-IV, 3-II, 4-I



A alternativa B está correta.

A fase 1 se relaciona com o processo II, pois trata da utilização de componentes prontos ou novos; a fase 2 com o processo III, que envolve a busca por serviços compatíveis; a fase 3 com o processo I, que aborda a adaptação de componentes; e a fase 4 com o processo IV, que indica o uso de modelos existentes.

Questão 5

Sobre a integração de componentes, a alternativa INCORRETA é:

A A fase de provisionamento dos componentes depende diretamente de tecnologia, pois define como os componentes serão adquiridos, localizados, reutilizados ou implementados.

B O processo UML Components lista possíveis maneiras de criar os componentes de software, aquisição, localização, reutilização e implementação de componentes.

C Na montagem do sistema, também dependente da tecnologia, é feita a implementação dos conectores e a ligação entre os componentes e os conectores do sistema.

D Na integração dos componentes é observada a adaptação e o comportamento dos componentes, requisitos de qualidade, disponibilidade, escalabilidade, confiabilidade, entre outros.

E Após a integração dos componentes, eles não podem ser utilizados como módulos separados em sistemas futuros.



A alternativa E está correta.

A afirmativa está incorreta porque, mesmo após a integração, os componentes mantêm sua independência e podem ser reutilizados como módulos em outros sistemas.

Questão 6

Na fase de integração de componentes, há um compromisso de implementar e integrar os componentes de forma que eles sejam consistentes com a documentação para facilitar o reuso dos mesmos no futuro.

Nesta atividade, são realizados os seguintes passos:

(I) De posse da documentação (modelos) dos componentes é definida qual a linguagem de implementação será utilizada.

(II) Após isso, é realizada a implementação dos componentes e suas respectivas interfaces.

(III) São realizados os testes dos componentes.

(IV) Uma vez implementados, e testados, os componentes são armazenados no repositório para possível utilização na implementação de uma aplicação.

Estão INCORRETAS as afirmações:

A I e II

B II e IV

C I e III

D II, III e IV

E Nenhuma afirmação está incorreta.



A alternativa E está correta.

Todas as afirmações estão corretas, pois refletem práticas válidas no contexto da engenharia de software orientada a componentes.

Complexidade de sistemas

A função do arquiteto de sistemas é abstrair toda a complexidade do sistema em elementos encapsulados, de maneira de que o usuário final tenha um sistema amigável e de fácil uso.

Ao mesmo tempo, esse sistema precisa resolver todos os problemas propostos, sem que o usuário perceba a complexidade embutida na aplicação.



Para o usuário, a solução deve parecer simples e fácil, já que toda a complexidade deve estar escondida e resolvida pelos desenvolvedores, na direção definida pelo arquiteto de sistemas.

A arquitetura de sistemas baseadas em componentes permite isso de maneira quase intuitiva.

Componentes de uma arquitetura de sistemas

Segundo Garlan (2000), do ponto de vista do arquiteto de sistemas, os seguintes componentes fazem parte de uma arquitetura de sistemas:

- Bancos de dados
- Servidores
- Clientes
- Filtros
- Um ou mais componentes

A interação entre eles pode ocorrer através de:

- Chamadas de procedimentos
- Acesso a variáveis

- Uso de protocolos para acesso a clientes e servidores
- Outros eventos quaisquer

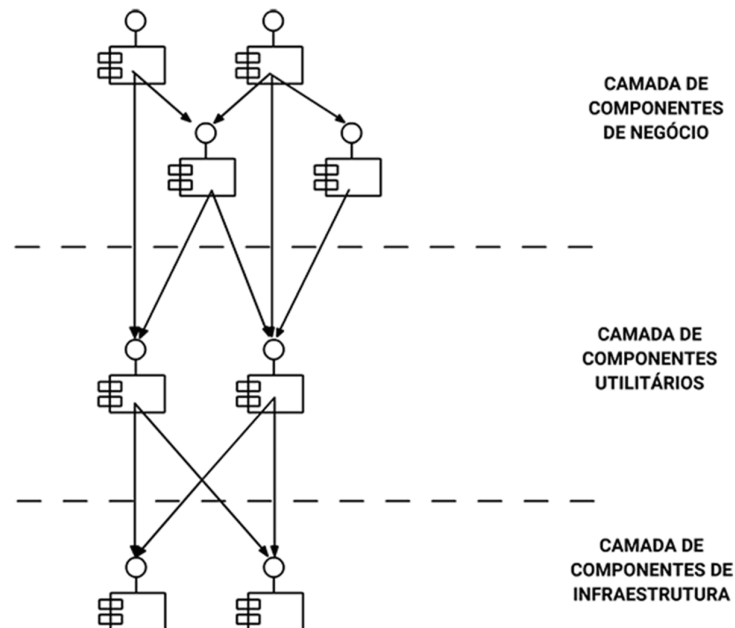
Divisão estrutural de componentes

A imagem representa a definição de camadas estruturais e sua importância para a arquitetura de sistemas.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



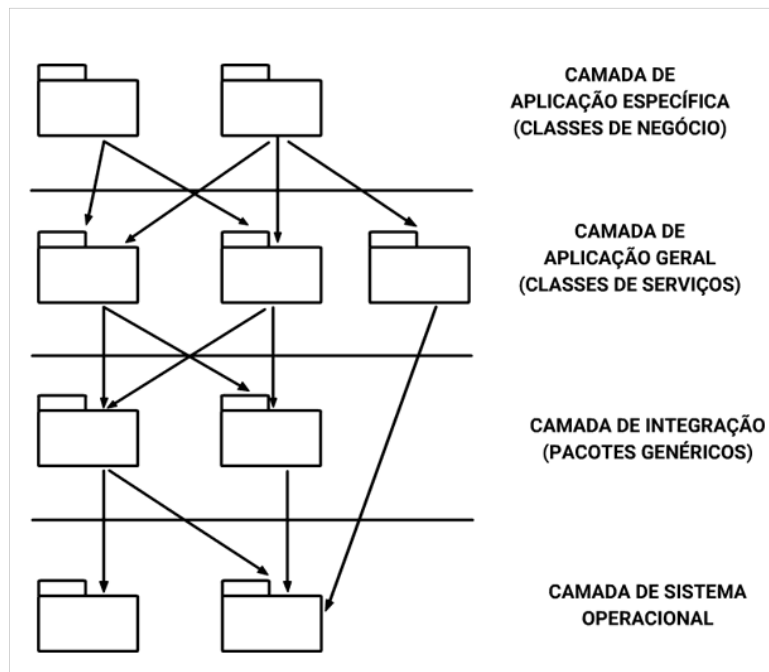
Os componentes de uma arquitetura de sistemas estão divididos em camadas estruturais, que, durante o desenvolvimento, devem ser consideradas de forma fundamental pelo arquiteto.

Essas camadas estruturais são os pilares do desenvolvimento de sistemas por componentes.

A partir delas é que se projeta a interação entre os componentes estruturais do sistema e se define a estrutura básica da arquitetura do sistema.

Camadas como elementos de controle da interação de componentes

Nesse contexto, os componentes são inseridos nas camadas que controlam a interação dos componentes e cada componente se comunica com as camadas vizinhas.



Exemplo de sistema de gerenciamento de versão

Para compreender melhor este assunto, é importante visualizar um exemplo de sistema desenvolvido na arquitetura de camadas:



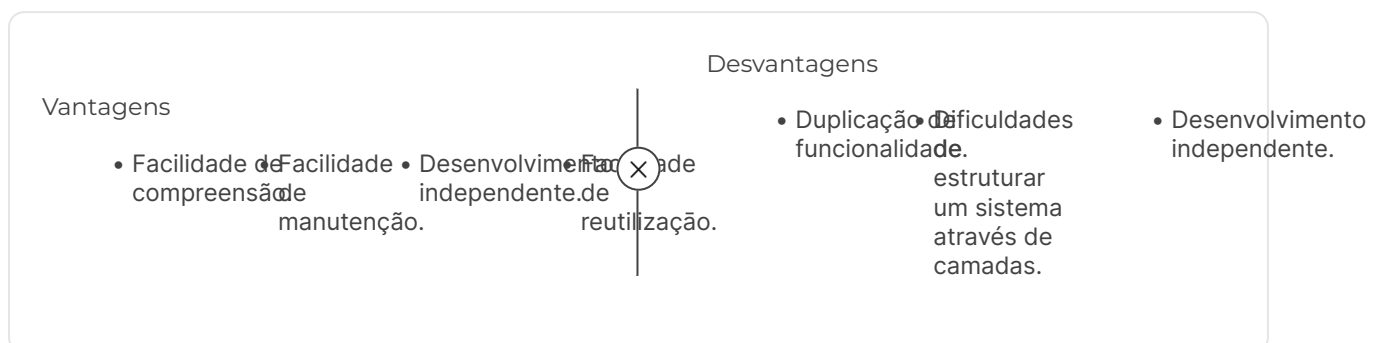
Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



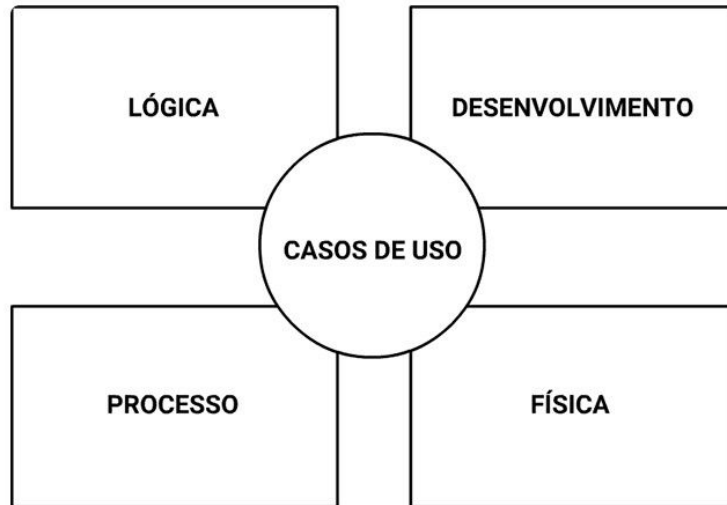
Vantagens e desvantagens do uso de camadas

Quando o arquiteto de sistemas decide utilizar camadas na arquitetura do sistema a ser desenvolvido, deve estar ciente dos fatores de vantagens e desvantagens desse tipo de abordagem.



Elementos da RUP (Rational Unified Process)

A arquitetura de sistemas envolve uma série de decisões que definem como o sistema será utilizado e apoiará a organização destinatária dele.



Esquema 4+1 da RUP.

Tem os seguintes objetivos:

- Definir os elementos e suas interfaces, de modo a estabelecer a estrutura do sistema;
- Estabelecer o comportamento associado entre o elementos que compõe o sistema;
- Analisar a composição desses elementos e suas estruturas, comportamentais e agregações de subsistemas.

Elementos de implementação que afetam a arquitetura de sistemas

Veja os seguintes elementos influenciam o desenvolvimento de sistemas baseados em componentes e que, por consequência, influenciam na sua integração:

- Arquitetura de computador.
- Linguagem de programação.
- Sistema operacional.
- Ambiente de interface gráfica.
- Necessidades de performance.

- Banco de dados.
- Bibliotecas de funções disponíveis.
- Portabilidade.
- Protocolos de rede.
- Sistemas legados.

Papel do arquiteto de sistemas na interação de componentes

O arquiteto de sistemas deve conhecer os seguintes elementos que compõem o desenvolvimento baseado em componentes:

- O negócio e os requisitos das aplicações a serem desenvolvidas;
- Os componentes disponíveis no ambiente e nos sistemas instalados;
- As tecnologias disponíveis para construção e arquitetura de sistemas;
- As metodologias de desenvolvimento adequadas ao sistema a ser desenvolvido.



Padrão arquitetura MVC (Model View Controller)

Quando o arquiteto de sistemas decide utilizar o padrão MVC para construir sua aplicação, deve levar em conta os seguintes aspectos:

Controller

Gerencia a interação da apresentação visual do sistema (View) com os comportamentos da aplicação (Model).

Model

Define a semântica da aplicação e define seu comportamento.

View

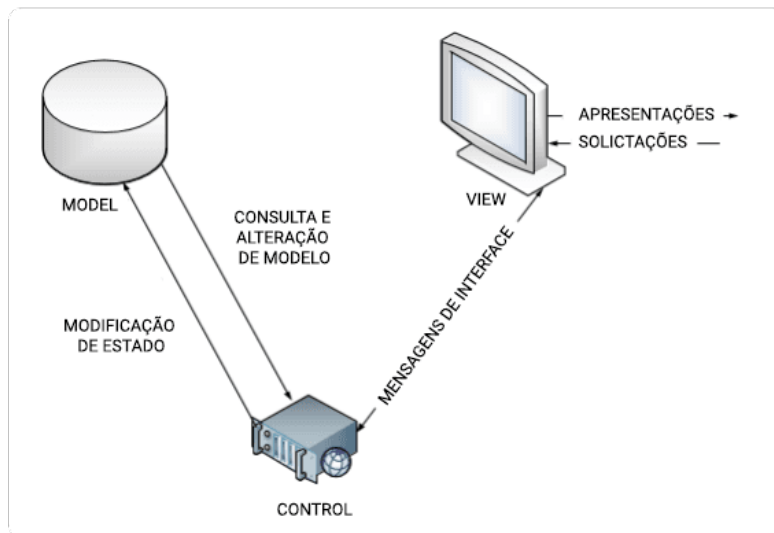
Define a apresentação visual da aplicação.

Agora, observe esses aspectos na imagem!



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Interação de componentes com AOS (Arquitetura Orientada a Serviços)

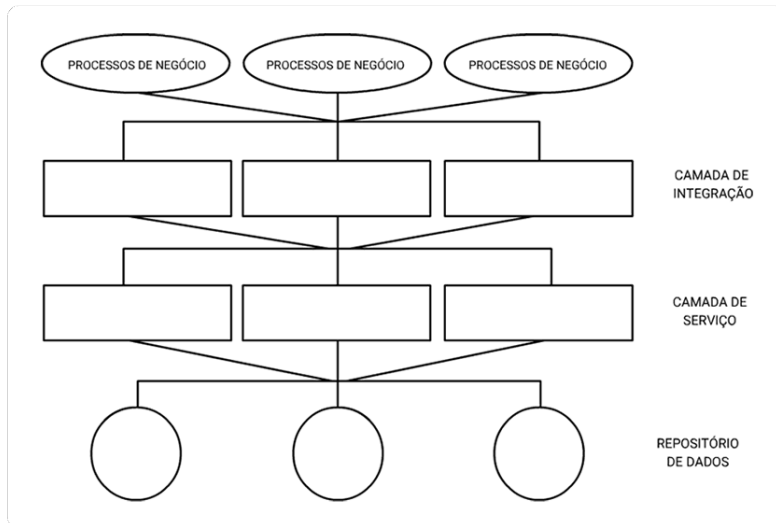
Na arquitetura de sistemas orientadas a serviços, todas as funcionalidades do sistema devem ser disponibilizadas na forma de serviços. Esses serviços se conectam através de um barramento de serviços (ESB).

Nesse barramento são disponibilizadas as interfaces na forma de web services que interagem com o repositório de dados.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Verificando o aprendizado

Sobre “chamada de procedimento/método”, é correto afirmar que:

- ☐ A Segue o modelo Cliente/Servidor.
- ☐ B Um componente que fornece uma interface com procedimentos/métodos para solicitar a execução de seus serviços é um servidor.
- ☐ C Componentes/aplicações que utilizam os serviços de outros componentes são seus clientes.
- ☒ D Todas as alternativas estão corretas.



A alternativa D está correta.

Todas as afirmativas estão corretas.

Considerações finais

Neste conteúdo, discutimos abordagens distintas no tratamento de requisitos no desenvolvimento de sistemas, destacando os extremos entre definições superficiais e exaustivas. Propusemos uma alternativa equilibrada, que reconhece a importância de requisitos bem definidos, mas defende um processo ágil, limitado a 5% do tempo total do projeto. Introduzimos a prototipação como estratégia eficiente para validação, permitindo ajustes rápidos e alinhamento com os objetivos do sistema.

A partir dessa base, exploramos o processo de identificação de componentes como etapa essencial dentro das boas práticas da arquitetura de sistemas. Apresentamos um conjunto de ações que orientam o arquiteto na análise e organização das estruturas, módulos e componentes, possibilitando a construção de modelos coerentes, escaláveis e adequados às necessidades do projeto.

Também analisamos como a correta identificação dos componentes influencia diretamente os resultados esperados, ressaltando a importância da definição clara das interfaces e das formas de interação entre eles. Esse cuidado contribui para a criação de soluções mais robustas, alinhadas com os princípios arquiteturais e capazes de atender a sistemas cada vez mais complexos.

Por fim, enfatizamos o papel estratégico do arquiteto de sistemas na condução desses processos e como sua atuação pode impactar positivamente na qualidade e na eficácia do produto final. Com o conhecimento adquirido, você estará mais preparado para enfrentar os desafios da modelagem por componentes e aplicar as práticas estudadas em projetos reais, promovendo melhores resultados no desenvolvimento de sistemas.

Explore +

Para saber mais sobre os assuntos estudados leia:

- O artigo "[Arquitetura de Sistemas de Informação - Evolução e Análise Comparativa de Modelos](#)".
- O material [Requisitos de Usuário e Requisitos do Sistema](#).

Acesse:

- O site da School of Computer Science e busque pelo termo "Software Architecture".
- Acesse o site DEVMEDIA e veja a "Metodologia de desenvolvimento de software".

Referências

- BASS, Len; CLEMENTS, Paul. **Software Architecture in Practice**. 2. ed. Nova Jersey: Pearson, 2003.

- CHESSMAN, John Daniels. **UML Components** – A Simple Process For Specifyng Component -Based Software. Boston: Addison-Wesley, 2000.
- PRADO LEITE, Julio Cesar Sampaio. **Engenharia de Requisitos**. Consultado na internet em 4 mai. 2016.
- RATIONAL SOFTWARE CORP. **Conceitos**: Tipos de Requisitos. Consultado na internet em 4 mai. 2016.
- SILVEIRA, Guilherme. **Introdução à arquitetura de design de software**. Rio de Janeiro: Elsevier (Livros Digitais), 2015.
- TERUEL, Evandro Carlos. **Arquitetura de sistemas para Web com Java Utilizando Design Patterns e Frameworks**. Rio de Janeiro: Ciência Moderna, 2012.